SQL Practice Questions (Based on Company2 Database)

Q.1. Show all employee records.

```
SELECT * FROM employees;
```

Q.2. Show names and salaries of all employees.

```
SELECT name, salary FROM employees;
```

Q.3. List employees working in the 'IT' department.

```
SELECT * FROM employees WHERE department = 'IT';
```

Q.4. Display employees with salary greater than 50000.

```
SELECT * FROM employees WHERE salary > 50000;
```

Q.5. List employees who joined in the year 2023.

```
SELECT * FROM employees WHERE YEAR(join_date) = 2023;
```

Q.6. Order employees by salary in descending order.

```
SELECT * FROM employees ORDER BY salary DESC;
```

Q.7. Count total number of employees.

```
SELECT COUNT(*) AS Total_Employees FROM employees;
```

Q.8. Get the average salary of employees.

```
SELECT AVG(salary) AS Average_Salary FROM employees;
```

Q.9. Find the highest and lowest salary.

```
SELECT MAX(salary) AS Highest, MIN(salary) AS Lowest FROM employees;
```

Q.10. Show employees with salary between 30000 and 60000.

```
SELECT * FROM employees WHERE salary BETWEEN 30000 AND 60000;
```

Q.11. Display employee names that start with 'R'.

```
SELECT * FROM employees WHERE name LIKE 'R%';
```

SQL Practice Questions (Based on Company2 Database)

Q.12. Show employees whose department is NULL.

```
SELECT * FROM employees WHERE department IS NULL;
```

Q.13. Show department-wise average salary.

```
SELECT department, AVG(salary) FROM employees GROUP BY department;
```

Q.14. Display number of employees in each department.

```
SELECT department, COUNT(*) FROM employees GROUP BY department;
```

Q.15. Get employees who joined before 2024.

```
SELECT * FROM employees WHERE join_date < '2024-01-01';
```

Q.16. Find names and joining dates of employees sorted by newest first.

```
SELECT name, join_date FROM employees ORDER BY join_date DESC;
```

Q.17. Update salary of 'Saif' to 70000.

```
UPDATE employees SET salary = 70000 WHERE name = 'Saif';
```

Q.18. Delete employee with position 'Student'.

```
DELETE FROM employees WHERE position = 'Student';
```

Q.19. List employees with position as 'Manager'.

```
SELECT * FROM employees WHERE position = 'Manager';
```

Q.20. Get unique departments.

```
SELECT DISTINCT department FROM employees;
```

Q.21. Show names and annual salary (salary * 12).

```
SELECT name, salary * 12 AS annual_salary FROM employees;
```

Q.22. List employees who joined in January.

```
SELECT * FROM employees WHERE MONTH(join_date) = 1;
```

SQL Practice Questions (Based on Company2 Database)

Q.23. Find employee with the maximum salary.

```
SELECT * FROM employees WHERE salary = (SELECT MAX(salary) FROM employees);
```

Q.24. Find employee with the minimum salary.

```
SELECT * FROM employees WHERE salary = (SELECT MIN(salary) FROM employees);
```

Q.25. Change 'Intern' position to 'Junior Developer'.

```
UPDATE employees SET position = 'Junior Developer' WHERE position = 'Intern';
```

Q.26. Display employees not in 'HR' department.

```
SELECT * FROM employees WHERE department <> 'HR';
```

Q.27. Get employee count for each position.

```
SELECT position, COUNT(*) FROM employees GROUP BY position;
```

Q.28. List top 3 highest paid employees.

```
SELECT * FROM employees ORDER BY salary DESC LIMIT 3;
```

Q.29. List employees in 'Cyber' or 'IT' departments.

```
SELECT * FROM employees WHERE department IN ('Cyber', 'IT');
```

Q.30. Count how many joined each year.

```
SELECT YEAR(join_date) AS join_year, COUNT(*) FROM employees GROUP BY YEAR(join_date);
```

Q.31. Display only the first 2 records.

```
SELECT * FROM employees LIMIT 2;
```

Q.32. List employees whose name ends with 'h'.

```
SELECT * FROM employees WHERE name LIKE '%h';
```

Q.33. Find employees whose name contains 'a'.

```
SELECT * FROM employees WHERE name LIKE '%a%';
```

SQL Practice Questions (Based on Company2 Database)

Q.34. Display salary after 5% hike for each employee.

```
SELECT name, salary, salary * 1.05 AS increased_salary FROM employees;
```

Q.35. Get total salary paid by department.

```
SELECT department, SUM(salary) FROM employees GROUP BY department;
```

Q.36. Find departments with more than 1 employee.

```
SELECT department FROM employees GROUP BY department HAVING COUNT(*) > 1;
```

Q.37. Find number of employees whose salary is over 60k.

```
SELECT COUNT(*) FROM employees WHERE salary > 60000;
```

Q.38. Show name in uppercase.

```
SELECT UPPER(name) FROM employees;
```

Q.39. Show first three letters of name.

```
SELECT SUBSTRING(name, 1, 3) FROM employees;
```

Q.40. Round off salary to nearest thousand.

```
SELECT name, ROUND(salary, -3) FROM employees;
```

Q.41. Add a new column showing bonus (10% of salary).

```
SELECT name, salary, salary * 0.10 AS bonus FROM employees;
```

Q.42. Find employees joined after 1st Jan 2025.

```
SELECT * FROM employees WHERE join_date > '2025-01-01';
```

Q.43. Find how many unique positions exist.

```
SELECT COUNT(DISTINCT position) FROM employees;
```

Q.44. Sort employees alphabetically by name.

```
SELECT * FROM employees ORDER BY name;
```

SQL Practice Questions (Based on Company2 Database)

Q.45. List employees who dont belong to any department.

```
SELECT * FROM employees WHERE department IS NULL;
```

Q.46. Find how many employees each year from 2020 to 2025.

```
SELECT YEAR(join_date), COUNT(*) FROM employees WHERE YEAR(join_date) BETWEEN 2020 AND 2025 GROUP BY YEAR(join_date);
```

Q.47. Check if there are any duplicate names.

```
SELECT name, COUNT(*) FROM employees GROUP BY name HAVING COUNT(*) > 1;
```

Q.48. Count employees who have 'Developer' in their position.

```
SELECT COUNT(*) FROM employees WHERE position LIKE '%Developer%';
```

Q.49. Try inserting a duplicate primary key (id = 1).

```
INSERT INTO employees (id, name, position, department, salary, join_date) VALUES (1, 'Duplicate', 'Tester', 'QA', 40000, '2025-04-01');
```

Q.50. Insert an employee with NULL name to test NOT NULL constraint.

```
INSERT INTO employees (position, department, salary, join_date) VALUES ('Developer', 'IT', 50000, '2025-04-01');
```

Q.51. Insert a negative salary to test CHECK constraint.

```
INSERT INTO employees (name, position, department, salary, join_date) VALUES ('NegativeSalary', 'Dev', 'IT', -10000, '2025-04-01');
```

Q.52. Create 'Customers' and 'Orders' tables with foreign key.

```
CREATE TABLE Customers (customer_id INT PRIMARY KEY, customer_name VARCHAR(100));
CREATE TABLE Orders (order_id INT PRIMARY KEY, order_date DATE, customer_id INT, FOREIGN KEY (customer_id) REFERENCES Customers(customer_id));
```

Q.53. Insert sample data into Customers and Orders.

```
INSERT INTO Customers VALUES (1, 'Alice'), (2, 'Bob'), (3, 'Charlie');
INSERT INTO Orders VALUES (101, '2025-01-01', 1), (102, '2025-01-02', 2);
```

Q.54. INNER JOIN: Show customers who placed orders.

```
SELECT c.customer_name, o.order_id FROM Customers c INNER JOIN Orders o ON c.customer_id = o.customer_id;
```

Q.55. LEFT JOIN: Show all customers and their orders (if any).

```
SELECT c.customer_name, o.order_id FROM Customers c LEFT JOIN Orders o ON c.customer_id = o.customer_id;
```

Q.56. RIGHT JOIN: Show all orders and their customer info.

```
SELECT c.customer_name, o.order_id FROM Customers c RIGHT JOIN Orders o ON c.customer_id = o.customer_id;
```

Q.57. Subquery: Show employees with salary above the average.

```
SELECT * FROM employees WHERE salary > (SELECT AVG(salary) FROM employees);
```

Q.58. Subquery: Find employee(s) with second highest salary.

```
SELECT * FROM employees WHERE salary = (SELECT MAX(salary) FROM employees WHERE salary < (SELECT MAX(salary) FROM employees));
```

Q.59. Subquery in FROM: Average salary by department using subquery.

```
SELECT department, avg_salary FROM (SELECT department, AVG(salary) AS avg_salary FROM employees GROUP BY department) AS dept_avg;
```

Q.60. Find departments with total salary > 100000.

```
SELECT department, SUM(salary) AS total FROM employees GROUP BY department HAVING SUM(salary) > 100000;
```

Q.61. Show employees who joined in the last 6 months.

```
SELECT * FROM employees WHERE join_date >= DATE_SUB(CURDATE(), INTERVAL 6 MONTH);
```

Q.62. Add a new column alias 'Full_Info' with name and position.

```
SELECT CONCAT(name, ' - ', position) AS Full_Info FROM employees;
```

Q.63. Show number of employees per month who joined.

```
SELECT MONTH(join_date) AS Month, COUNT(*) AS Total FROM employees GROUP BY MONTH(join_date);
```

SQL Practice Questions (Based on Company2 Database)

Q.64. Find employees who have not been assigned a department.

```
SELECT * FROM employees WHERE department IS NULL OR department = '';
```

Q.65. Show total salary per position, only for positions with >1 employee.

```
SELECT position, SUM(salary) FROM employees GROUP BY position HAVING COUNT(*) > 1;
```

Q.66. Show employees whose name length is more than 6.

```
SELECT * FROM employees WHERE LENGTH(name) > 6;
```

Q.67. Get top 2 highest salaries in each department.

```
SELECT * FROM (SELECT *, DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rnk FROM employees) AS ranked WHERE rnk <= 2;
```

Q.68. Check which employees share the same salary.

```
SELECT salary, COUNT(*) FROM employees GROUP BY salary HAVING COUNT(*) > 1;
```